Lab 11: Claude guesses a field and submits without validating

Topic 1: Agentic Architecture & Orchestration · Task 1.4: Implement multi-step workflows with enforcement and handoff patterns

Runs on Simulator 6: Structured Data Extraction · Self-practice

Source: SkillCertPro practice exam Part 1, Q12 (unofficial third-party question)

The question

The system exposes client-executed tools named extract_fields , validate_record , submit_record , and create_review_ticket . Production logs show that when a required field is ambiguous, Claude sometimes guesses a value and emits submit_record without a successful validate_record result. The application then executes the requested tool call, and downstream systems receive records that should have gone to review. What change would most effectively enforce the required workflow?

A. Stop the agent loop whenever the assistant text includes phrases such as “ready to submit”.

B. Add stronger system instructions telling Claude to create review tickets for ambiguous required fields.

C. Add an application-level gate that blocks submit_record unless validate_record has succeeded for the same record.

D. Set tool_choice to the validate_record tool whenever the extraction prompt mentions ambiguous fields.

Simulate it live

```
# In the browser: open Simulator 6, then Lab 11
cd labs
uv run serve.py

# In Claude Code, from the labs folder
/lab 11

# In a terminal, from simulators/simulator-6-structured-extraction/
uv run extract.py --case UC-12
```

What you should see

DOC-07 is blocked at submit_record, nothing submits unvalidated, and a review ticket opens.

evidence/cases/UC-12.json lists at least one blocked submit_record (DOC-07), zero submits executed without a prior validation success, and a review ticket for the ambiguous document. The loop ends on end_turn, never on text matching.

Open `simulators/simulator-6-structured-extraction/evidence/` and find it yourself.

The answer

C. An application-level gate blocks submit_record until validation succeeds.

With client tools, Claude only requests a call. Your application decides whether and how to execute it, so that boundary is the right place to enforce business-critical ordering such as “validate before submit”. tool_choice steers whether Claude may, must or must not use tools on one turn, but it does not prove later calls meet application prerequisites. Prompts guide behaviour but cannot match the guarantee of refusing to execute an invalid request. Drive the loop from structured signals: execute allowed calls while stop_reason == "tool_use" , return tool_result blocks, and stop at end_turn .

Why the others fail

A. Parsing natural-language phrases is an unreliable control signal, and it does not block submit_record .

B. Probabilistic. The model is already skipping the intended path.

D. Forces one call on one turn and depends on spotting “ambiguous” in the prompt. A later unvalidated submit_record can still run.

Check yourself

- Learner can point to the line in the gate that checks validated[record_id] before executing submit_record.
- Learner explains why the gate returns is_error: true rather than silently dropping the call.
- Learner states why tool_choice: tool for validate_record would not have been sufficient.

Now change one thing

Change DOC-04's printed total in fixtures/invoices.txt, rerun, and watch by_document_type.invoice fall while the others hold.

Third-party practice material from SkillCertPro, reconstructed from a video transcript in the Tertiary Markdown Vault. Not a real exam item. Wording may differ from the original.